

Lab Sheet 3

Name: Nitin Panwar

Section: B

Enrollment No.: 39

```
In [1]: import cv2 import
numpy as np
import matplotlib.pyplot as plt import
urllib.request import os
print("Libraries imported
successfully!")
```

Libraries imported successfully!

```
In [3]: image_url =
"https://raw.githubusercontent.com/opencv/opencv/master/sampl
image_path = "lena.jpg" urllib.request.urlretrieve(image_url,
image_path) print("Dataset downloaded successfully!")
```

Dataset downloaded successfully!

```
In [4]: image = cv2.imread(image_path)

if image is None:
    print("Error: Image not loaded")
else:
    print("Image loaded successfully!")
    print("Image Shape:", image.shape)
```

Image loaded successfully!

Image Shape: (512, 512, 3)

```
In [5]: gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

print("Image converted to grayscale!")
print("Grayscale Shape:", gray.shape)
```

Image converted to grayscale!

Grayscale Shape: (512, 512)

```
In [6]: gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

print("Image converted to grayscale!")
print("Grayscale Shape:", gray.shape)
```

Image converted to grayscale!

Grayscale Shape: (512, 512)

```
In [7]: plt.figure(figsize=(8, 6))

plt.imshow(gray, cmap="gray")
plt.title("Original Grayscale Image")
plt.axis("off")
```

```
plt.show()
```

Original Grayscale Image



```
In [8]: def add_salt_pepper_noise(image, amount=0.05):
        noisy = image.copy()

        # Salt noise
        num_salt = int(amount * image.size / 2)

        coords = [
            np.random.randint(0, i, num_salt)
            for i in image.shape
        ]

        noisy[coords[0], coords[1]] = 255

        # Pepper noise
        num_pepper = int(amount * image.size / 2)

        coords = [
            np.random.randint(0, i, num_pepper)
            for i in image.shape
        ]

        noisy[coords[0], coords[1]] = 0

        return noisy
```

```
noisy_image = add_salt_pepper_noise(gray)

print("Salt-and-pepper noise added successfully!")
```

Salt-and-pepper noise added successfully!

```
In [9]: plt.figure(figsize=(8, 6))

plt.imshow(noisy_image, cmap="gray")
plt.title("Image with Salt-and-Pepper Noise") plt.axis("off") plt.show()
```

Image with Salt-and-Pepper Noise



```
In [10]: gaussian = cv2.GaussianBlur(noisy_image, (5, 5), 0)

plt.figure(figsize=(8, 6))
plt.imshow(gaussian,
cmap="gray") plt.title("Gaussian
Filter") plt.axis("off")
plt.show()
```

Gaussian Filter



```
In [11]: median = cv2.medianBlur(noisy_image, 5)

plt.figure(figsize=(8, 6))
plt.imshow(median,
cmap="gray") plt.title("Median
Filter") plt.axis("off")
plt.show()
```

Median Filter



```
In [12]: average = cv2.blur(noisy_image, (5, 5))

plt.figure(figsize=(8, 6))
plt.imshow(average, cmap="gray")
plt.title("Average / Mean  
Filter") plt.axis("off")
plt.show()
```

Average / Mean Filter



```
In [13]: laplacian = cv2.Laplacian(gray, cv2.CV_64F)

laplacian = cv2.convertScaleAbs(laplacian)

plt.figure(figsize=(8, 6))
plt.imshow(laplacian, cmap="gray")
plt.title("Laplacian High-Pass
Filter") plt.axis("off") plt.show()
```

Laplacian High-Pass Filter

